Credit Card Approval Prediction

Project Documentation

Abstract

Credit Card Approval Prediction is a machine learning based web application that automates the decision of whether a credit card application is likely to be approved or rejected. The system is trained on historical applicant data covering financial and demographic attributes, and evaluates new applications using a trained classification model. Four algorithms were compared during development — Logistic Regression, Random Forest, XGBoost, and Decision Tree — and the best performing model was integrated into a Flask web application. The solution is designed to be deployed on the cloud through IBM Watson Machine Learning, enabling scalable and real-time predictions through a simple, user-friendly interface called ApprovalIQ.

Problem Statement

Banks and financial institutions receive thousands of credit card applications every day. A significant portion of these are rejected due to factors such as high existing loan balances, insufficient income levels, or excessive credit inquiries. Manually reviewing each application is time-consuming and prone to human error, making the process inefficient at scale. This project addresses that gap by using a trained machine learning model to instantly evaluate an applicant's financial and demographic profile, helping analysts prioritize applications and helping prospective customers understand their eligibility before formally applying.

Features

- Instant approval or rejection prediction based on applicant financial and demographic details.
- Four classification algorithms evaluated — Logistic Regression, Random Forest, XGBoost, and Decision Tree — with the best model deployed.
- Simple web interface (ApprovalIQ) for entering personal and financial details.
- Feature engineering pipeline that converts multi-class payment status codes into binary risk labels.
- Batch screening support for compliance officers reviewing high-risk applicants.
- Cloud-ready deployment through IBM Watson Machine Learning for scalable, real-time predictions.
- Clear result screen showing an Approved or Not Approved outcome with next-step actions.

Tech Stack

- Frontend: HTML, CSS
- Backend: Python, Flask
- Machine Learning: Scikit-learn, Pandas, NumPy

- Data Visualization: Matplotlib, Seaborn
- Database: SQLite
- Development Tools: Anaconda Navigator, PyCharm
- Deployment: IBM Cloud, Render
- Version Control: Git, GitHub
- Dataset Source: Kaggle (Credit Card Approval dataset)

Installation Steps

1. Clone the project repository from GitHub to your local machine.
2. Create a new Python environment using Anaconda Navigator or venv.
3. Install the required dependencies listed in requirements.txt using pip.
4. Ensure the trained model file is present in the project directory.
5. Run the Flask application locally using python app.py.
6. Open the local server address shown in the terminal in a web browser.

Usage Instructions

1. Open the ApprovalIQ home page and click the "Check Eligibility" button.
2. Enter the applicant's personal details, including gender, age, family status, and family members count.
3. Enter the applicant's financial details, including annual income, income type, years employed, and education type.
4. Submit the form to send the details to the trained model.
5. View the instant prediction result, shown as Application Approved or Application Not Approved.
6. Use "Try Another Prediction" to test another applicant, or "Back to Home" to return to the main page.

Screenshots

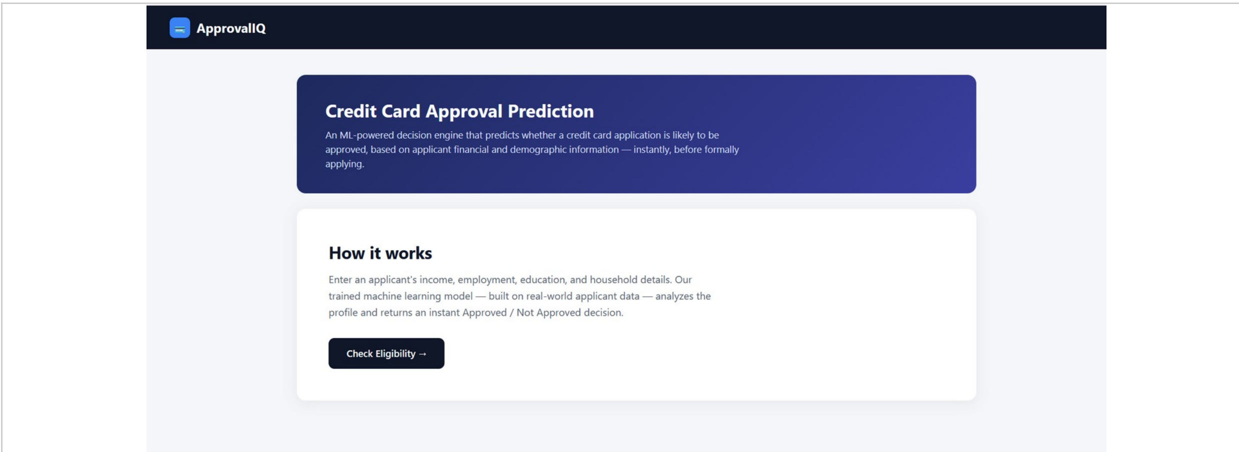

Figure 1: ApprovalIQ home page introducing the Credit Card Approval Prediction tool.

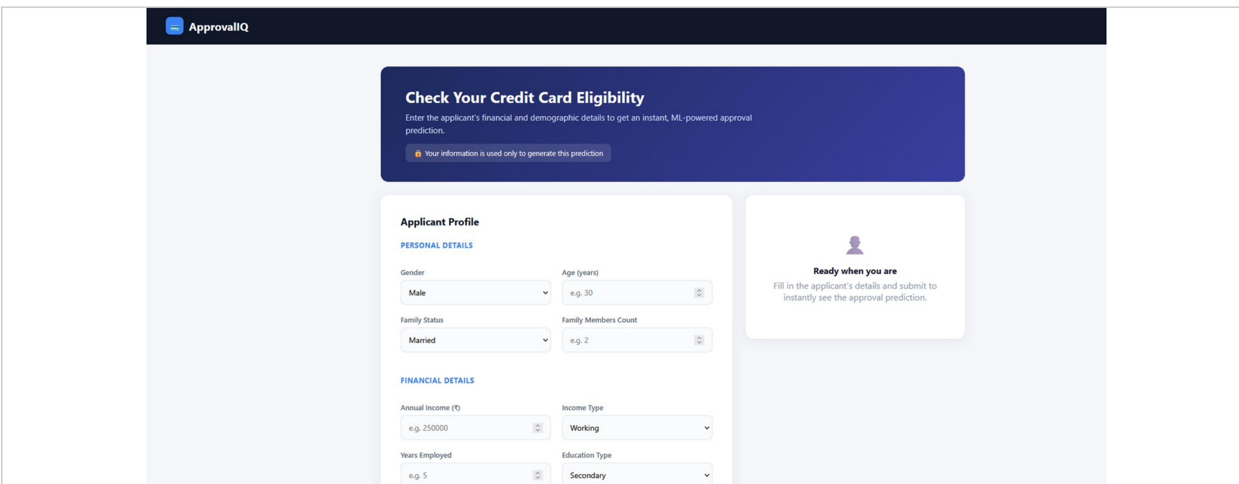

Figure 2: Applicant profile form for entering personal and financial details.

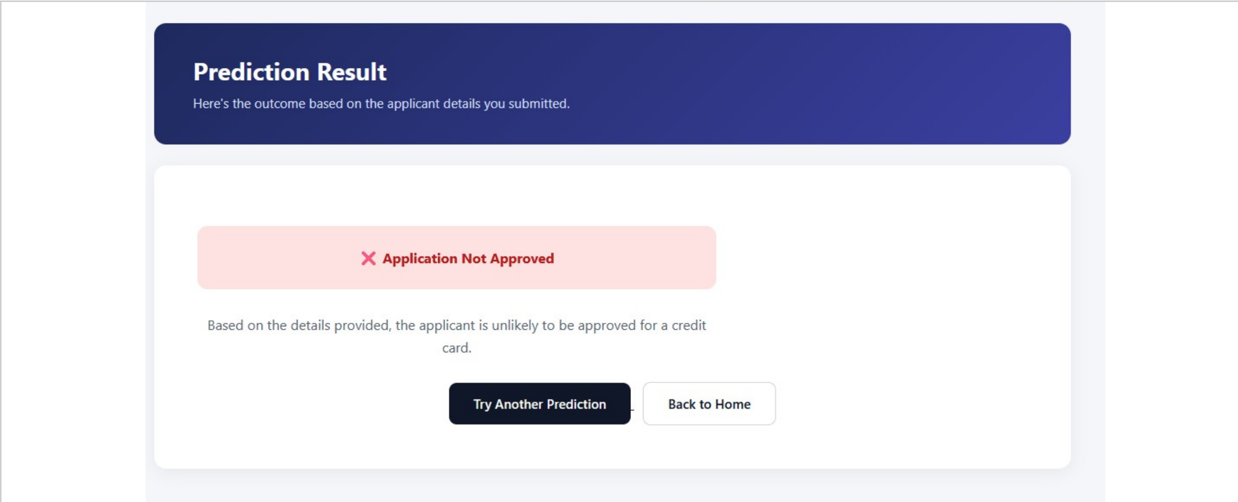

Figure 3: Prediction result screen showing an application that was not approved.

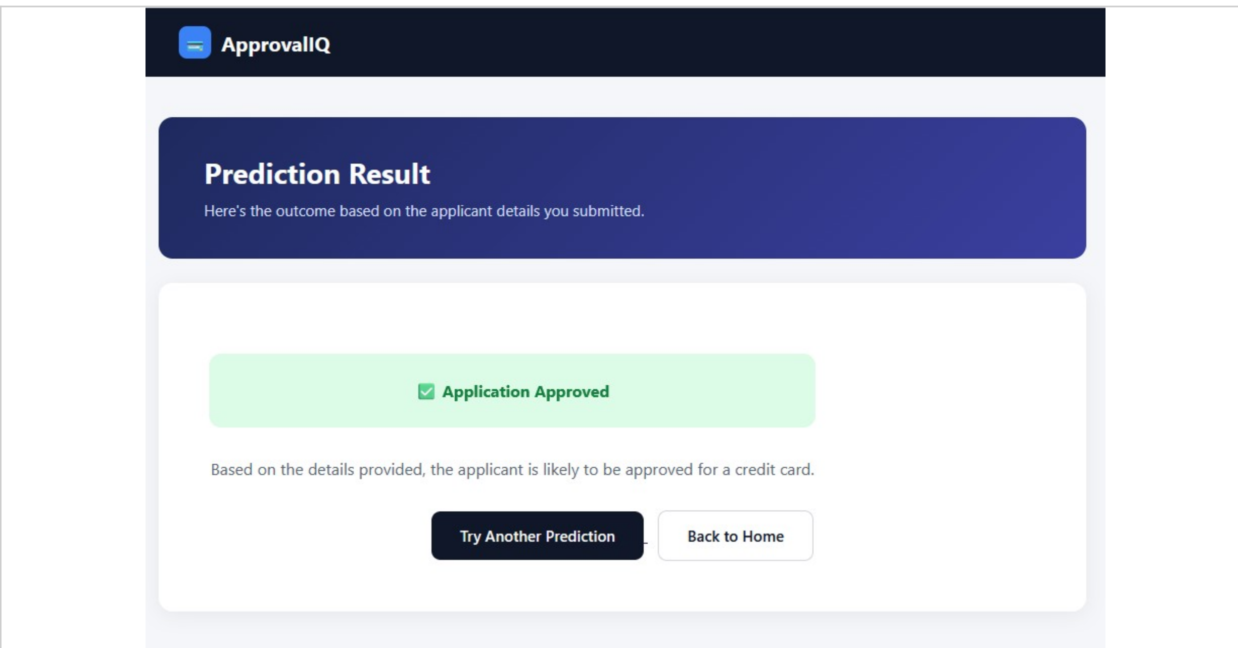

Figure 4: Prediction result screen showing an application that was approved.

Conclusion:

The application is developed using Flask and a Logistic Regression model trained on historical credit card applicant data. Users enter applicant details through the web interface, the input is preprocessed using the saved scaler and encoders, and the trained model predicts whether the application is likely to be approved or rejected. The application is deployed on Render, enabling real-time access through a web browser.